

04. Spring Boot Application Properties

4.1. Giới thiệu về Application Properties

Spring Boot Application Properties là một hệ thống cấu hình mạnh mẽ cho phép bạn tùy chỉnh hành vi của ứng dụng mà không cần thay đổi mã nguồn. Thay vì hardcode các giá trị cấu hình trong code, bạn có thể đặt chúng trong các file cấu hình bên ngoài, giúp ứng dụng trở nên linh hoạt và dễ dàng triển khai trên các môi trường khác nhau (development, testing, production).

Spring Boot hỗ trợ nhiều định dạng file cấu hình khác nhau, trong đó phổ biến nhất là:

- **Properties format** (`.properties`): Định dạng key-value đơn giản, truyền thống của Java.
- **YAML format** (`.yml` hoặc `.yaml`): Định dạng có cấu trúc phân cấp, dễ đọc và viết hơn.

4.2. Cấu trúc và vị trí của Application Properties

4.2.1. Vị trí file cấu hình

Spring Boot tự động tìm kiếm các file cấu hình theo thứ tự ưu tiên sau:

1. `application.properties` hoặc `application.yml` trong thư mục `config/` của thư mục làm việc hiện tại
2. `application.properties` hoặc `application.yml` trong thư mục làm việc hiện tại
3. `application.properties` hoặc `application.yml` trong thư mục `config/` của classpath (thường là `src/main/resources/config/`)
4. `application.properties` hoặc `application.yml` trong classpath root (thường là `src/main/resources/`)

File có độ ưu tiên cao hơn sẽ ghi đè các thuộc tính trong file có độ ưu tiên thấp hơn.

4.2.2. So sánh Properties và YAML format

Properties format (`application.properties`):

Plain Text

```
# Server configuration
server.port=8080
server.servlet.context-path=/api

# Database configuration
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=user
spring.datasource.password=password
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA configuration
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect

# Logging configuration
logging.level.root=INFO
logging.level.com.example.myapp=DEBUG
logging.file.name=app.log
```

YAML format (`application.yml`):

YAML

```
# Server configuration
server:
  port: 8080
  servlet:
    context-path: /api

# Database configuration
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
    username: user
    password: password
    driver-class-name: com.mysql.cj.jdbc.Driver

# JPA configuration
jpa:
  hibernate:
    ddl-auto: update
```

```

show-sql: true
properties:
  hibernate:
    dialect: org.hibernate.dialect.MySQL8Dialect

# Logging configuration
logging:
  level:
    root: INFO
    com.example.myapp: DEBUG
  file:
    name: app.log

```

YAML format thường được ưa chuộng hơn vì:

- Cấu trúc phân cấp rõ ràng, dễ đọc
- Ít lặp lại tiền tố
- Hỗ trợ comments tốt hơn
- Có thể định nghĩa arrays và objects phức tạp

4.3. Các nhóm cấu hình quan trọng

4.3.1. Server Configuration

YAML

```

server:
  port: 8080           # Cổng server (mặc định: 8080)
  address: 0.0.0.0     # Địa chỉ IP bind (mặc định: tất cả
interfaces)
  servlet:
    context-path: /api # Context path của ứng dụng
    session:
      timeout: 30m     # Thời gian hết hạn session
  compression:
    enabled: true      # Bật nén response
    mime-types: text/html,text/xml,text/plain,text/css,application/json
  ssl:
    enabled: false     # Bật HTTPS
    key-store: classpath:keystore.p12

```

```
key-store-password: password
key-store-type: PKCS12
```

4.3.2. Database Configuration

YAML

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
    username: ${DB_USERNAME:user}      # Sử dụng biến môi trường với giá
    # giá trị mặc định
    password: ${DB_PASSWORD:password}
    driver-class-name: com.mysql.cj.jdbc.Driver
    hikari:
      # HikariCP connection pool settings
      maximum-pool-size: 20
      minimum-idle: 5
      connection-timeout: 30000
      idle-timeout: 600000
      max-lifetime: 1800000

  # JPA/Hibernate Configuration
  jpa:
    hibernate:
      ddl-auto: validate                # none, validate, update, create,
    # create-drop
    naming:
      physical-strategy:
        org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
    show-sql: false                    # Hiển thị SQL queries trong log
    properties:
      hibernate:
        dialect: org.hibernate.dialect.MySQL8Dialect
        format_sql: true
        use_sql_comments: true
      jdbc:
        batch_size: 20
        order_inserts: true
        order_updates: true
```

4.3.3. Logging Configuration

YAML

```

logging:
  level:
    root: INFO
    org.springframework.web: DEBUG
    org.hibernate.SQL: DEBUG
    org.hibernate.type.descriptor.sql.BasicBinder: TRACE
    com.example.myapp: DEBUG

  pattern:
    console: '%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n'
    file: '%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n'

  file:
    name: logs/application.log
    max-size: 10MB
    max-history: 30
    total-size-cap: 1GB

```

4.3.4. Security Configuration

YAML

```

spring:
  security:
    user:
      name: admin
      password: admin123
      roles: ADMIN
    oauth2:
      client:
        registration:
          google:
            client-id: your-google-client-id
            client-secret: your-google-client-secret
            scope: openid,profile,email

```

4.3.5. Actuator Configuration

YAML

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,env,beans
      base-path: /actuator
  endpoint:
    health:
      show-details: when-authorized
    info:
      enabled: true
  info:
    env:
      enabled: true
```

4.4. Profile-specific Properties

Spring Boot cho phép bạn tạo các file cấu hình riêng cho từng profile (môi trường):

- `application-dev.yml` : Cấu hình cho môi trường development
- `application-test.yml` : Cấu hình cho môi trường testing
- `application-prod.yml` : Cấu hình cho môi trường production

Ví dụ `application-dev.yml` :

YAML

```
spring:
  datasource:
    url: jdbc:h2:mem:testdb
    username: sa
    password:
    driver-class-name: org.h2.Driver

  h2:
    console:
      enabled: true

  jpa:
    hibernate:
      ddl-auto: create-drop
```

```
show-sql: true
```

```
logging:  
  level:  
    com.example.myapp: DEBUG
```

Ví dụ `application-prod.yml` :

YAML

```
spring:  
  datasource:  
    url: jdbc:mysql://prod-db-server:3306/proddb  
    username: ${DB_USERNAME}  
    password: ${DB_PASSWORD}  
  
  jpa:  
    hibernate:  
      ddl-auto: validate  
    show-sql: false  
  
logging:  
  level:  
    root: WARN  
    com.example.myapp: INFO  
  file:  
    name: /var/log/myapp/application.log
```

Để kích hoạt profile:

Bash

```
# Thông qua command line  
java -jar myapp.jar --spring.profiles.active=prod  
  
# Thông qua biến môi trường  
export SPRING_PROFILES_ACTIVE=prod  
  
# Thông qua application.properties  
spring.profiles.active=dev
```

4.5. Custom Properties và @ConfigurationProperties

Bạn có thể định nghĩa các thuộc tính tùy chỉnh và ánh xạ chúng vào các lớp Java:

application.yml:

YAML

```
myapp:
  name: My Spring Boot Application
  version: 1.0.0
  features:
    - authentication
    - logging
    - monitoring
  database:
    pool-size: 10
    timeout: 30s
  security:
    jwt:
      secret: mySecretKey
      expiration: 86400
  cors:
    allowed-origins:
      - http://localhost:3000
      - http://localhost:4200
    allowed-methods:
      - GET
      - POST
      - PUT
      - DELETE
```

Java Configuration Class:

Java

```
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;
import java.time.Duration;
import java.util.List;

@Component
@ConfigurationProperties(prefix = "myapp")
public class AppProperties {

    private String name;
    private String version;
    private List<String> features;
```



```

private Database database = new Database();
private Security security = new Security();

// Getters and Setters
public String getName() { return name; }
public void setName(String name) { this.name = name; }

public String getVersion() { return version; }
public void setVersion(String version) { this.version = version; }

public List<String> getFeatures() { return features; }
public void setFeatures(List<String> features) { this.features =
features; }

public Database getDatabase() { return database; }
public void setDatabase(Database database) { this.database = database; }

public Security getSecurity() { return security; }
public void setSecurity(Security security) { this.security = security; }

// Nested classes
public static class Database {
    private int poolSize;
    private Duration timeout;

    // Getters and Setters
    public int getPoolSize() { return poolSize; }
    public void setPoolSize(int poolSize) { this.poolSize = poolSize; }

    public Duration getTimeout() { return timeout; }
    public void setTimeout(Duration timeout) { this.timeout = timeout; }
}

public static class Security {
    private Jwt jwt = new Jwt();
    private Cors cors = new Cors();

    // Getters and Setters
    public Jwt getJwt() { return jwt; }
    public void setJwt(Jwt jwt) { this.jwt = jwt; }

    public Cors getCors() { return cors; }
    public void setCors(Cors cors) { this.cors = cors; }

    public static class Jwt {
        private String secret;
        private long expiration;
    }
}

```

```

        // Getters and Setters
        public String getSecret() { return secret; }
        public void setSecret(String secret) { this.secret = secret; }

        public long getExpiration() { return expiration; }
        public void setExpiration(long expiration) { this.expiration =
expiration; }
    }

    public static class Cors {
        private List<String> allowedOrigins;
        private List<String> allowedMethods;

        // Getters and Setters
        public List<String> getAllowedOrigins() { return allowedOrigins;
}

        public void setAllowedOrigins(List<String> allowedOrigins) {
this.allowedOrigins = allowedOrigins; }

        public List<String> getAllowedMethods() { return allowedMethods;
}

        public void setAllowedMethods(List<String> allowedMethods) {
this.allowedMethods = allowedMethods; }
    }
}

```

Sử dụng trong Service:

Java

```

@Service
public class AppInfoService {

    private final AppProperties appProperties;

    public AppInfoService(AppProperties appProperties) {
        this.appProperties = appProperties;
    }

    public String getAppInfo() {
        return String.format("App: %s v%s, Features: %s",
            appProperties.getName(),
            appProperties.getVersion(),
            appProperties.getFeatures());
    }
}

```

```

    public boolean isFeatureEnabled(String feature) {
        return appProperties.getFeatures().contains(feature);
    }
}

```

4.6. Validation cho Configuration Properties

Bạn có thể thêm validation cho các configuration properties:

Java

```

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;
import org.springframework.validation.annotation.Validated;

import javax.validation.constraints.*;
import java.util.List;

@Component
@ConfigurationProperties(prefix = "myapp")
@Validated
public class AppProperties {

    @NotBlank(message = "App name cannot be blank")
    @Size(min = 3, max = 50, message = "App name must be between 3 and 50 characters")
    private String name;

    @Pattern(regexp = "\\d+\\.\\d+\\.\\d+", message = "Version must follow semantic versioning (x.y.z)")
    private String version;

    @NotEmpty(message = "At least one feature must be enabled")
    private List<String> features;

    @Valid
    private Database database = new Database();

    // Getters and Setters...

    public static class Database {
        @Min(value = 1, message = "Pool size must be at least 1")
        @Max(value = 100, message = "Pool size cannot exceed 100")
        private int poolSize;
    }
}

```

```
@NotNull(message = "Timeout cannot be null")
private Duration timeout;

// Getters and Setters...
}
}
```

4.7. Bản chất của Application Properties

Bản chất của Spring Boot Application Properties là cung cấp một cơ chế mạnh mẽ và linh hoạt để quản lý cấu hình ứng dụng theo nguyên tắc "Externalized Configuration". Điều này cho phép bạn tách biệt hoàn toàn cấu hình khỏi mã nguồn, giúp cùng một bản build có thể chạy trên nhiều môi trường khác nhau chỉ bằng cách thay đổi cấu hình. Hệ thống này hỗ trợ nhiều nguồn cấu hình với thứ tự ưu tiên rõ ràng, từ file properties/yaml đến biến môi trường và command line arguments, giúp đáp ứng mọi nhu cầu triển khai từ đơn giản đến phức tạp. Khả năng type-safe configuration thông qua `@ConfigurationProperties` và validation tích hợp giúp đảm bảo tính đúng đắn của cấu hình ngay từ khi ứng dụng khởi động, góp phần quan trọng vào việc xây dựng các ứng dụng production-ready.